



دانشگاه علم و صنعت
دانشکده ریاضی و علوم کامپیوتر

مدرس: دکتر الهی
طراحی و توسعه سامانه تعاملی مدیریت، پالایش و تحلیل داده‌های موسیقی
(Spotify Data Studio)

۱ هدف پروژه (Project Vision)

هدف نهایی این پروژه، ساخت یک «جعبه‌ابزار مهندسی و تحلیلی داده (Data Engineering Toolkit)» به صورت ماژولار و شیء‌گرایی است. در دنیای واقعی، داده‌های خام (مثل اطلاعات میلیون‌ها آهنگ در اسپاتیفای) به دلیل خطاهای سیستمی یا انسانی، پر از نقص، مقادیر گم‌شده (Missing) و داده‌های پرت (Outliers) هستند. کدهای آماتور و خطی (Scripting) توانایی مدیریت این حجم از داده را ندارند و با کوچک‌ترین خطایی کرش می‌کنند. شما در این پروژه باید برنامه‌ای بسازید که:

۱. داده‌های خام پلتفرم اسپاتیفای را به طور ایمن از دیسک بخواند و آن‌ها را در قالب اشیای نرم‌افزاری کپسوله‌سازی کند.

۲. با طراحی یک معماری شیء‌گرایی مستحکم (ارث‌بری و چندریختی)، ماژول‌های پیشرفته‌ای برای تمیزکاری و اعمال الگوریتم‌های آماری روی داده‌ها خلق کند.

۳. یک داشبورد تعاملی کنترل‌شده (CLI) در اختیار کاربر بگذارد تا بتواند داده‌ها را دستکاری کند، آهنگ جدید اضافه کند و گزارش‌های تحلیلی و نمودارهای آماری دقیقی از رفتار لایک و ترندهای موسیقی استخراج کند.

۲ دیتاست پروژه

لینک دیتاست پیشنهادی پروژه در گروه درس ارسال خواهد شد. این دیتاست شامل اطلاعات و ویژگی‌های صوتی بیش از ۱۰۰,۰۰۰ آهنگ از پلتفرم اسپاتیفای است. نکته مهم: شما مجاز هستید به جای دیتاست پیشنهادی، از یک دیتاست دلخواه دیگر استفاده کنید؛ اما دیتاست انتخابی شما باید حداقل دارای ۲۰ ویژگی (ستون) و حداقل ۱۰۰,۰۰۰ نمونه (سطر) باشد. ویژگی‌های موجود در دیتاست پیشنهادی به شرح زیر است:

```
track_id, artists, album_name, track_name, popularity, duration_ms,  
explicit, danceability, energy, key, loudness, mode, speechiness,  
acousticness, instrumentalness, liveness, valence, tempo, time_signature,  
track_genre
```

۳ ساختار پوشه‌بندی پروژه (Directory Structure)

ساختار پیشنهادی زیر برای ماژولار کردن پروژه طراحی شده است. کدهای شما به هیچ وجه نباید خطی و در قالب یک اسکریپت آشفته باشند. با این حال، شما در تغییر این ساختار و اضافه کردن فایل‌ها یا پکیج‌های جدید کاملاً آزاد هستید، مشروط بر اینکه اصول شیء‌گرایی و نظم کدنویسی رعایت شود.

```
spotify_studio/  
  
  data/  
    spotify_tracks.csv  
  
  src/  
    __init__.py  
    data_loader.py  
    data_cleaner.py  
    data_analyzer.py  
    data_visualizer.py  
  
  main.py
```

۴ نیازمندی‌های فنی و معماری نرم‌افزار

۱.۴ کلاس Song (کپسوله‌سازی و اعتبارسنجی)

باید کلاسی به نام Song تعریف کنید که هر آهنگ در سیستم یک نمونه (Instance) از آن باشد. تمام ویژگی‌های دیتاست باید در این کلاس کپسوله‌سازی شوند. با استفاده از Property Setter ها، باید مطمئن شوید داده‌ی ورودی معتبر است (مثلاً popularity باید بین ۰ تا ۱۰۰ و ویژگی‌های صوتی مانند danceability و energy بین ۰ تا ۱ باشند، در غیر این صورت برنامه باید یک ValueError مناسب را raise کند).

۲.۴ ماژول data_cleaner.py (گسترش ارث‌بری و تنوع الگوریتمی)

در این بخش باید ساختار چندریختی (Polymorphism) را به طور کامل پیاده‌سازی کنید. برای تمیزکاری داده‌ها، سیستم شما باید مجهز به چندین روش مختلف باشد:

• بخش داده‌های گم‌شده (Missing Values):

- کلاس پایه BaseImputer با متد انتزاعی impute().
- زیرکلاس‌های MeanImputer (پر کردن با میانگین) و MedianImputer (پر کردن با میانه).
- زیرکلاس KNNImputer (پر کردن هوشمند با استفاده از الگوریتم همسایگان نزدیک با کمک کتابخانه scikit-learn).

• بخش داده‌های پرت (Outliers):

- کلاس پایه BaseOutlierHandler با متد انتزاعی handle().
- زیرکلاس IQROutlierHandler (شناسایی با روش دامنه میان‌چارکی).
- زیرکلاس ZScoreOutlierHandler (شناسایی با استفاده از نمره استاندارد $Z - score$ و انحراف معیار).

۳.۴ ماژول `data_loader.py` (کار با فایل)

متد `append_song(song: Song)` باید آهنگ جدید را در حالت **Append Mode** (بدون بازنویسی کل فایل) به انتهای فایل CSV اضافه کند و همزمان دیتای موجود در حافظه (RAM) را بروزرسانی کند تا در تحلیل‌های بعدی اثرگذار باشد.

۴.۴ مدیریت استثناها (Exception Handling)

برنامه تحت هیچ شرایطی نباید در اثر ورودی اشتباه کاربر یا خطاهای سیستمی (مانند `FileNotFoundError` یا `ValueError`) کرش کند، بلکه باید پیغام مناسب نمایش داده و به منوی اصلی بازگردد.

۵ منوی تعاملی کاربر (CLI Dashboard)

با اجرای فایل `main.py` باید منوی تعاملی طراحی شده در ترمینال به کاربر نمایش داده شود.

```
===== Spotify Data Studio & Management System =====
1. Load Dataset & View Missing Values Report
2. Clean Missing Values (Mean / Median / KNN)
3. Handle Outliers (IQR / Z-Score)
4. Add a New Song to the Dataset (Interactive Input)
5. Calculate Genre Insights & Correlation Matrix
6. Generate Advanced Visualizations (Plots)
7. Exit
=====
Enter your choice (1-7): _
```

۶ بخش خلاقیت و توسعه آزاد (امتیازی باز)

شما می‌توانید با تکیه بر خلاقیت خود، قابلیت‌های جدیدی به پروژه اضافه کنید که در صورت پیاده‌سازی اصولی، نمره امتیازی ویژه‌ای به همراه خواهد داشت. چند نمونه ایده برای خلاقیت: روش‌های پالایش جدید، تحلیل‌های آماری پیشرفته، رسم نمودارهای خلاقانه (مثل واژه‌ابر یا نمودارهای رادار برای ویژگی‌های صوتی یک سبک خاص) یا هر ابزار خلاقانه دیگری که این سامانه را کارآمدتر کند.

۷ بخش امتیازی: پیاده‌سازی مدل یادگیری ماشین (اختیاری)

در صورت تمایل می‌توانید ماژول `data_modeler.py` را به پروژه اضافه کنید. در این بخش کلاسی به نام `SongPredictor` طراحی می‌شود که با استفاده از کتابخانه `scikit-learn`، یک مدل دسته‌بندی (مانند Decision Tree یا KNN) را آموزش می‌دهد تا بر اساس ۴ ویژگی صوتی ورودی، محبوبیت آهنگ (بالای ۵۰ به عنوان Hit و زیر ۵۰ به عنوان Flop) را پیش‌بینی کند. در این بخش، اگر کاربر قبل از آموزش دادن مدل تمایل به پیش‌بینی داشت، برنامه باید خطای مناسبی را `raise` کند تا سیستم کرش نکند.

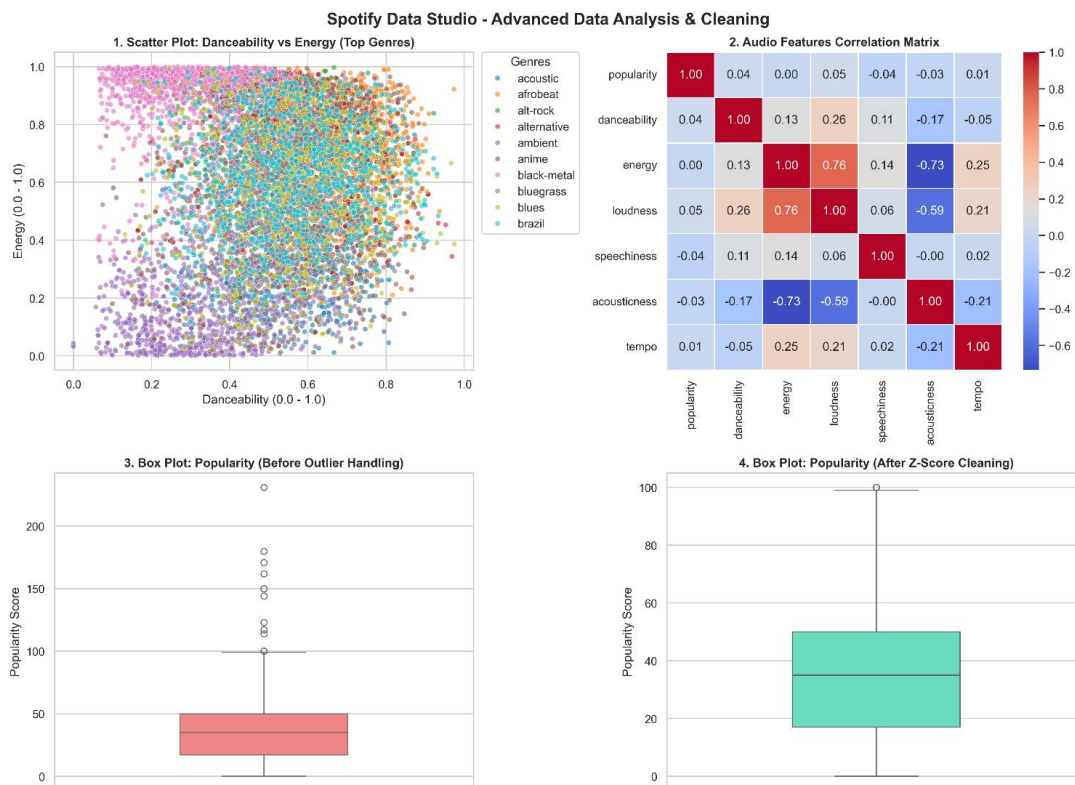
۸ گزارش تحلیلی عمیق و مستندات پروژه (وزن: ۴۰٪ نمره کل)

با توجه به اینکه ابزارهای هوش مصنوعی قادر به درک و شهود تجربه شخصی و تحلیل دقیق داده‌های شما نیستند، این بخش وزن بالایی دارد و معیار اصلی سنجش اصالت پروژه است. شما موظفید یک گزارش مکتوب (فایل PDF حداکثر در ۵ صفحه) شامل موارد زیر تحویل دهید:

۱. کالبدشکافی چالش‌های فنی و منطقی (Post-Mortem): ذکر حداقل ۲ چالش بزرگ که در طول کدنویسی یا پیاده‌سازی منطق‌های ریاضی (مثل فرمول $Z - score$ یا هماهنگ‌سازی KNN با شیء‌گرایی) با آن مواجه شدید و روش حل آن.

۲. تجربیات یادگیری: توضیح کاربرد واقعی مفاهیم OOP و جلوگیری از تکرار کد (اصل DRY) در این پروژه.

۳. تحلیل داستان‌سرایی داده‌ها (Data Storytelling): قرار دادن اسکرین‌شات حداقل ۴ نمودار خروجی برنامه (Box Plot، Scatter Plot) قبل و بعد از تمیزکاری، و Heatmap همبستگی) همراه با تحلیل عمیق ساختار داده‌ها.



۹ نحوه بارم‌بندی و قانون ضریب تسلط

بارم‌بندی بخش‌های مختلف پروژه به صورت ضریب بخش‌بندی شده است. نمره کامل مبنای پروژه متعاقباً اعلام خواهد شد. ضرایب بخش‌ها با تمرکز بر پیاده‌سازی درست منطق‌ها و تحلیل‌ها به شرح زیر است:

بخش‌های پروژه	ضریب نمره	معیارهای ارزیابی
اصول شیء‌گرایی و مهندسی نرم‌افزار	۰.۴	طراحی درست کلاس‌ها، ارث‌بری و کپسوله‌سازی
صحت منطق‌ها و پایداری برنامه (CLI)	۰.۲	پیاده‌سازی درست فرمول‌ها و هندل خطا (Z-score/IQR)، کار با فایل
گزارش تحلیلی و مستندات چالش‌ها	۰.۴	کیفیت اسکرین‌شات‌ها، عمق تحلیل نمودارها، اصالت متن
بخش امتیازی (مدل هوش مصنوعی)	+۰.۲	پیاده‌سازی کامل کلاس مدل‌ساز، آموزش و پیش‌بینی بدون کرش
بخش خلاقیت و توسعه آزاد	امتیازی	میزان پیچیدگی فنی و عمق ایده خلاقانه جدید

۱.۹ فرمول محاسبه نمره نهایی (ضریب تسلط)

ارائه این پروژه الزامی است و نحوه برگزاری آن متعاقباً اعلام خواهد شد. شما تا زمان قفل نمرات برای تحویل و ارائه فرصت دارید. میزان تسلط شما در زمان ارائه به عنوان یک ضریب مستقیم در نمره‌ی کل ضرب خواهد شد. نمره نهایی شما طبق فرمول زیر محاسبه می‌شود:

$$Score_{Final} = [(Score_{Code} \times 1.0) + Score_{Bonus}] \times \left(\frac{Mastery\%}{100} \right)$$

به عنوان مثال، اگر دانشجو نمره کامل بخش‌های کد و گزارش را کسب کند (۱۰۰ نمره) اما درصد تسلط او در زمان ارائه و پاسخ به سوالات ۷۰٪ ارزیابی شود، نمره نهایی او در سیستم ۷۰ ثبت خواهد شد.